

Tecnologías de Producción de Software 2026

Opción Ruby

Clase 2

Métodos, bloques e iteradores

Facultad de Informática — Universidad Nacional de La Plata

Viernes 11/09/2026

Objetivos de la clase

- Escribir métodos que reciban una cantidad variable de parámetros (*args) y usar el operador splat para invocarlos con un arreglo.
- Comprender qué es un bloque, cómo se escribe, cómo se recibe con yield y qué reglas rigen el alcance de sus variables.
- Diferenciar un bloque implícito de un Proc explícito, y combinar bloques con manejo de excepciones.
- Incorporar el vocabulario de la iteración en Ruby: each, map, select, reduce, y su variante lazy, incluyendo su uso sobre archivos.
- Congelar objetos con freeze y reconocer el error que produce intentar modificarlos.
- Crear jerarquías de clases con herencia simple, identificando qué se hereda de una superclase.
- Diseñar una clase (Palabra) que valide sus datos con excepciones propias y ofrezca métodos de consulta basados en iterar sus caracteres.

Métodos con número variable de parámetros

- ***numeros** agrupa cualquier cantidad de argumentos posicionales en un único Array, dentro del método.
- Es distinto de recibir un solo arreglo como parámetro: acá se invoca con los valores sueltos, sin armar la colección a mano.
- Es habitual escribir primero la versión que recibe un arreglo, y después una variante más flexible con *args.

```
def ordenar_arreglo(arr)
  arr.sort
end
ordenar_arreglo([1,4,6,2,3,0,10])
# => [0, 1, 2, 3, 4, 6, 10]

def ordenar(*numeros)
  numeros.sort
end
ordenar(1,4,6,2,3,5,0,10,9)
# => [0,1,2,3,4,5,6,9,10]
```

El operador splat (*) también sirve para invocar

- Si ya tenés un arreglo y el método espera parámetros sueltos (*args), pasarlo directamente no “desempaca” sus elementos.
- **El mismo símbolo *** cumple dos roles distintos: agrupa argumentos en la definición del método, y los despliega en la invocación.
- El error no siempre se nota de inmediato: el método puede ejecutarse sin excepción, pero con un resultado incorrecto.

```
entrada = [10,9,1,2,3,5,7,8]
```

```
# Forma incorrecta:
```

```
ordenar(entrada)
```

```
# => [[10,9,1,2,3,5,7,8]]
```

```
# Forma correcta:
```

```
ordenar(*entrada)
```

```
# => [1,2,3,5,7,8,9,10]
```

Parámetros de cualquier tipo, con #to_s

- *args permite aceptar objetos de cualquier tipo, no solo de uno — el método no necesita saber de antemano qué le van a pasar.
- **#to_s** es lo que permite tratar de forma uniforme objetos tan distintos como un entero, un Hash o un Symbol (visto en la Clase 1).
-

```
def longitud(*args)
  args.each do |a|
    puts "\"#{a.to_s}\" --> #{a.to_s.length}"
  end
end

longitud(9, 'Hola', {un:'hash'}, :ruby)
# "9" --> 1
# "Hola" --> 4
# {:un=>"hash"} --> 13
# ruby --> 4
```

Bloques: qué son y cómo se escriben

- Un bloque es código encerrado entre llaves `{ }` o entre las palabras clave `do / end`.
- Ambas formas son idénticas en significado, y difieren solo en la precedencia con la que se asocian a la invocación.
- **Convención:** cuando el bloque entra en una sola línea se usa `{ }`; cuando ocupa más de una línea se usa `do / end`.
- Los bloques pueden verse como métodos anónimos: código que se guarda para ejecutarse más adelante, no en el momento en que aparece escrito.
- En Ruby, un bloque solo puede usarse inmediatamente después de invocar un método — nunca de forma aislada.

```
# Una línea: llaves
3.times { |i| puts i }

# Mas de una línea: do / end
3.times do |i|
  puts i
  puts i * i
end
```

Bloques: yield y block_given?

- Un bloque es código anónimo que se pasa junto a la invocación de un método, entre { } o do...end.
- **yield** ejecuta el bloque recibido; **block_given?** indica si efectivamente se pasó alguno.
- A diferencia de un parámetro común, el bloque no aparece en la firma del método: es información implícita.

```
def repetir_saludo
  return "sin bloque" unless block_given?
  yield
end

repetir_saludo { "Hola!" }
# => "Hola!"

repetir_saludo
# => "sin bloque"
```

da_nil?: un bloque como argumento implícito

- da_nil? recibe un bloque, lo ejecuta con yield, y pregunta si el valor que devolvió es nil.
- El bloque puede ser vacío ({ }): en Ruby eso también es una expresión válida, cuyo valor es nil.
- Es el ejemplo mínimo de “bloque como argumento implícito de comportamiento”, la base de todo lo que sigue.

```
def da_nil?  
  yield.nil?  
end  
  
da_nil? { }  
# => true  
  
da_nil? do  
  'Algo distinto de nil'  
end  
# => false
```

Bloques explícitos: Proc y procesar_hash

- **Un Proc** es un objeto de primera clase: se puede guardar en una variable, pasar como parámetro explícito, y ejecutar con `#call`.
- `procesar_hash` arma un nuevo Hash invirtiendo clave y valor, aplicando el Proc recibido a cada clave original.
- **-> (arrow)** es la forma concisa de escribir una lambda, una variante de Proc.

```
def procesar_hash(hash, proc)
  nuevo = {}
  hash.each do |k, v|
    nuevo[v] = proc.call(k)
  end
  nuevo
end

hash = {'clave'=>1, otra_clave: 'valor'}
procesar_hash(hash, ->(x){ x.to_s.upcase })
# => {1=>"CLAVE", "vaLor"=>"OTRA_CLAVE"}
```

Bloques: alcance de variables

- **Regla importante:** si dentro de un bloque aparece una variable con el mismo nombre que una variable ya existente en el alcance externo, ambas son la misma variable.
- El bloque no crea automáticamente su propia copia: puede leer y también reescribir el valor de una variable definida afuera.
- Esto puede producir errores silenciosos cuando el bloque, sin querer, pisa una variable que se sigue usando después del bloque.
- Si una variable aparece únicamente dentro del bloque (como el acumulador de un cálculo interno), es local a él.

```
figura = "circulo" # fuera del bloque
suma = 0
[1,2,3,4].each do |valor|
  figura = valor * valor # ¡misma variable!
  suma += figura
end

figura.upcase # BOOM: figura ahora
              # es un Integer
```

Bloques: variables locales al bloque

- Ruby permite declarar explícitamente que una o más variables sean locales al bloque, aunque su nombre coincida con una del alcance externo.
- Se agregan después de un punto y coma (;), luego de los parámetros normales del bloque: **|param; local|**
- Esa variable local no “pisa” la de afuera: al terminar el bloque, la de afuera conserva su valor original.

```
figura = "circulo"
suma = 0
[1,2,3,4].each do |valor; figura|
  figura = valor * valor # otra variable
  suma += figura
end

puts suma    # => 30
puts figura  # => "circulo"
```

Bloques y manejo de excepciones

- Un método puede envolver la ejecución de un bloque en `begin/rescue` para reaccionar distinto según la clase de la excepción.
- **rescue `ClassError`** captura solo esa clase (y sus subclases); `rescue` sin clase captura `StandardError`.
- **raise** sin argumentos, dentro de un `rescue`, relanza la excepción actual — reaccionás sin “tragarte” el error.

```
def ejecutar_seguro(*args)
  yield(*args)
  :ok
rescue RuntimeError
  puts "Hay algo mal..."; :rt
rescue NoMethodError => e
  puts "Y este método? #{e.message}"
  :nm
rescue => e
  puts "Y ahora?"; raise
end
```

Iteradores: each, map, select, reduce

- Todas las colecciones que incluyen el módulo Enumerable (Array, Hash, Range...) comparten estos métodos de iteración.
- **each** recorre y devuelve la colección original; **map** transforma cada elemento en uno nuevo.
- **select** (o filter) conserva lo que cumple el bloque; **reduce** (o inject) combina todo en un único valor acumulado.

```
[1,2,3].map { |n| n * 2 }  
# => [2, 4, 6]
```

```
[1,2,3,4,5,6].select(&:even?)  
# => [2, 4, 6]
```

```
[1,2,3,4].reduce(:+)  
# => 10
```

Bloques y archivos

- File.open abre un archivo y devuelve un objeto que también responde a each, igual que un arreglo: cada línea del archivo es un elemento a iterar.
- Combinar each con un bloque y una expresión regular es una forma muy compacta de filtrar líneas sin escribir un FOR ni un WHILE.
- **line =~ /patrón/** dentro del bloque decide, línea por línea, si esa línea se imprime.

```
File.open("/etc/passwd").each do |line|  
  puts line if line =~ /root/  
end
```

```
# => root:x:0:0:root:/root:/bin/bash
```

Enumeradores lazy

- Un enumerador no-lazy como map o select procesa y materializa toda la colección de inmediato, aunque solo necesites unos pocos resultados.
- **.lazy** encadena las mismas operaciones sin ejecutarlas todavía: arma una “receta” que recién se evalúa cuando pedís algo concreto (first, take, force).
- Es imprescindible para secuencias infinitas o muy grandes, evitando calcular de más.

```
(1..Float::INFINITY)
  .lazy
  .select(&:even?)
  .first(5)
# => [2, 4, 6, 8, 10]

(1..Float::INFINITY).lazy
  .map { |n| n * 2 }
  .select { |n| n % 3 == 0 }
  .first(3)
# => [6, 12, 18]
```

Freezando objetos

- **#freeze** vuelve un objeto inmutable: cualquier intento posterior de modificarlo lanza `FrozenError`.
- Freezar no crea una copia: si otra variable apunta al mismo objeto (por asignación simple, no por `dup` ni `clone`), también queda afectada — porque es literalmente el mismo objeto.
- **#equal?** compara identidad de objeto (mismo `object_id`), a diferencia de `==` que compara contenido.
- Se conecta con lo visto en la Clase 1: dos strings con igual contenido no son el mismo objeto, pero dos variables que apuntan al mismo objeto sí.

```
persona1 = "Tim"
persona2 = persona1 # mismo objeto
persona1.freeze
persona2[0] = "J"

# FrozenError: can't modify
# frozen String: "Tim"

persona1.equal?(persona2) # => true
```

Herencia

- Permite crear clases que son un refinamiento o especialización de otra clase.
- **Subclase:** la clase que se refina. **Superclase:** la clase original. También se usan los términos clase hija y clase padre.
- El mecanismo es simple: se heredan las capacidades de la clase padre.
- Los métodos de instancia y de clase del padre quedan disponibles automáticamente en los hijos, sin reescribirlos.

Herencia: ejemplo

- **class Child < Parent** declara que Child hereda de Parent: el símbolo < expresa esa relación.
- Child no define say_hello, pero lo obtiene igual: lo hereda de Parent sin necesidad de copiar código.
- Este es el mecanismo básico que la Práctica 2 retoma para modelar jerarquías de clases más completas.

```
class Parent
  def say_hello
    puts "Hello from #{self.class}"
  end
end

class Child < Parent
end

Parent.new.say_hello # Hello from Parent
Child.new.say_hello  # Hello from Child
```

Caso de estudio: la clase Palabra

- El constructor valida el String recibido y define dos excepciones propias: NoEsUnaPalabra y EsUnStringVacio.
- Una excepción personalizada es, simplemente, una clase que hereda de StandardError.
- **raise ClaseError, "mensaje"** crea la instancia de la excepción y la lanza en un solo paso.

```
class NoEsUnaPalabra < StandardError; end
class EsUnStringVacio < StandardError; end

class Palabra
  def initialize(texto)
    if texto.strip.empty?
      raise EsUnStringVacio, "Es un string vacío"
    elsif texto =~ /\s/
      raise NoEsUnaPalabra,
        "#{texto} no es una palabra"
    end
    @texto = texto
  end
end
```

La clase Palabra: métodos de consulta

- Los métodos de consulta se apoyan en iterar los caracteres: chars, select y uniq son suficientes para vocales y consonantes.
- **es_panvocalica?** y **es_palindroma?** son predicados que devuelven booleano, siguiendo la convención de nombres de la Clase 1.
- **"murcielago".vocales** => ["u","i","e","a","o"] → **es_panvocalica?** => true

```
VOCALES = 'aeiou'

def vocales
  @texto.downcase.chars
    .select { |c| VOCALES.include?(c) }.uniq
end

def es_panvocalica?
  vocales.sort == VOCALES.chars.sort
end

def es_palindroma?
  @texto.length >= 3 &&
    @texto.downcase == @texto.downcase.reverse
end
```